

Esercizi:

INDUZIONE STRUTTURALE

CASO BASE $\Rightarrow$ lo dimostro mostrando che vale sull'Elemento che nella definizione è all'Insieme

PASSO $\Rightarrow$ devo Applicare Prima L'IPOTESI INDUTTIVA

$\rightarrow$ Suppongo Quello che devo dimostrare a degli Elementi più Complessi rispetto al CASO BASE

devo Riuscire a Ricondurmi alle forme delle FORMULA che devo DIMOSTRARE (USARE VARIABILI PER RAGGRUPPARE)

SCOPING STATICO Ambiente di RIFERIMENTO Stabilito a **Tempo di Compilazione** (Quello che CONTIENE definizione della Chiamata) Quindi è indipendente da dove si fa la CHIAMATA, **è Sempre UGUALE**.

Gestisco Tramite la Catena Statica

① Fare Schema Annidamento e Calcolare $Sd()$ per TUTTI

② Catena delle Chiamate

③ Link Dinamico è Sempre All' RDA Precedente

④ Calcolare $K_{chiamato} = Sd(chiamante) - Sd(chiamato) + 1$ $\forall$ chiamato

⑤ Per Variabili non locali Calcolare:

$$N_{var} = Sd(Procedura\ che\ lo\ usa) - Sd(Più\ Vicina\ che\ la\ definisce)$$

→ #Volte che devo risalire la CATENA STATICA da P per Trovare il Blocco Contiene la var di RIFERIMENTO

⑥

Ricordarsi che se una Procedura Termina non va Ricopiato RDA

SCOPING DINAMICO L'AMBIENTE DI RIFERIMENTO È QUELLO VALIDO
al **MOMENTO DELLA CHIAMATA DELLA PROCEDURA** (POTENZIALMENTE DIVERSO
OGNI VOLTA)

Gestisco Tramite **CRT** (Tabella Centrale dei Riferimenti)

- ① faccio un ARRAY con tutte le Variabili che Vengono USATE
- ② Ogni Volta che Viene Sovrascritta una definizione di una var.
Aggiungo in Testa Alla Coda
- ③ Posso Sempre farmi lo Schema di ANNIDAMENTO per Capire
che Procedura definisca Cosa.

Nel caso in cui ci fosse una **Procedura passata Come PARAM**
che ha dei RIFERIMENTI non Locali allora devo Anche analizzare
le **REGOLE DI BINDING:**

① **SHALLOW** ⇒ "SCOPING DINAMICO" per le PROCEDURE, fa riferim.
all' Ultimo AMBIENTE Creato (**QUELLO CHE FA**
LA CHIAMATA)

② **DEEP** ⇒ "SCOPING STATICO" per le PROCEDURE, si UTILIZZA l'
ambiente che **CREA IL LEGAME STATICO**

RICORSIONE

① Dare DEFINIZIONE

↳ Ricorsiva SSE **Richiama SE STESSA**

② Dire Quando è RICORSIONE DI CODA

↳ Solo se viene RITORNATO il Valore della chiamata
Ricorsiva **Senza Ulteriori Rielaborazioni o**

Computazioni

↳ Utile perché AUMENTA EFFICIENZA dello SPAZIO USATO

↳ Costruisce il RISULTATO ad ogni Chiamata e alla fine (giunto Alla BASE) LO RESTITUISCE.

③ Dire Che Cosa fa (fare TEST)

④ Proporre RC + Chiamata D'Inizializzazione.

⑤ Simulare le Chiamate per DIMOSTRARE il Risultato

Ricorsione in Code ha SEMPRE più Parametri Rispetto Alla Ricorsione Classica

Mi Serve allocazione **DINAMICA della MEMORIA**, Visto che non so a priori Quanta Memoria mi Serve e Quante Volte Veccà Chiamata la PROCEDURA RICORSIVA. Ma Anche L'IMPLEMENTAZIONE delle procedure per permetterla

COMPLETAMENTO CODICE

① Deve Essere ESEGUIBILE Quindi tutto Quello che si usa deve Essere DICHIARATO

② devo Avere un AMBIENTE NON LOCALE

→ Stessa Variabile Sia
Nell'Ambiente di Chiamata che in
Quello di DEFINIZIONE.

SEMANTICA COMANDI / ISTRUZIONI

DINAMICA:

- /// Valuta l'Espressione
- /// Elabora la dichiarazione
- /// Trasformazione della MEMORIA

STATICA

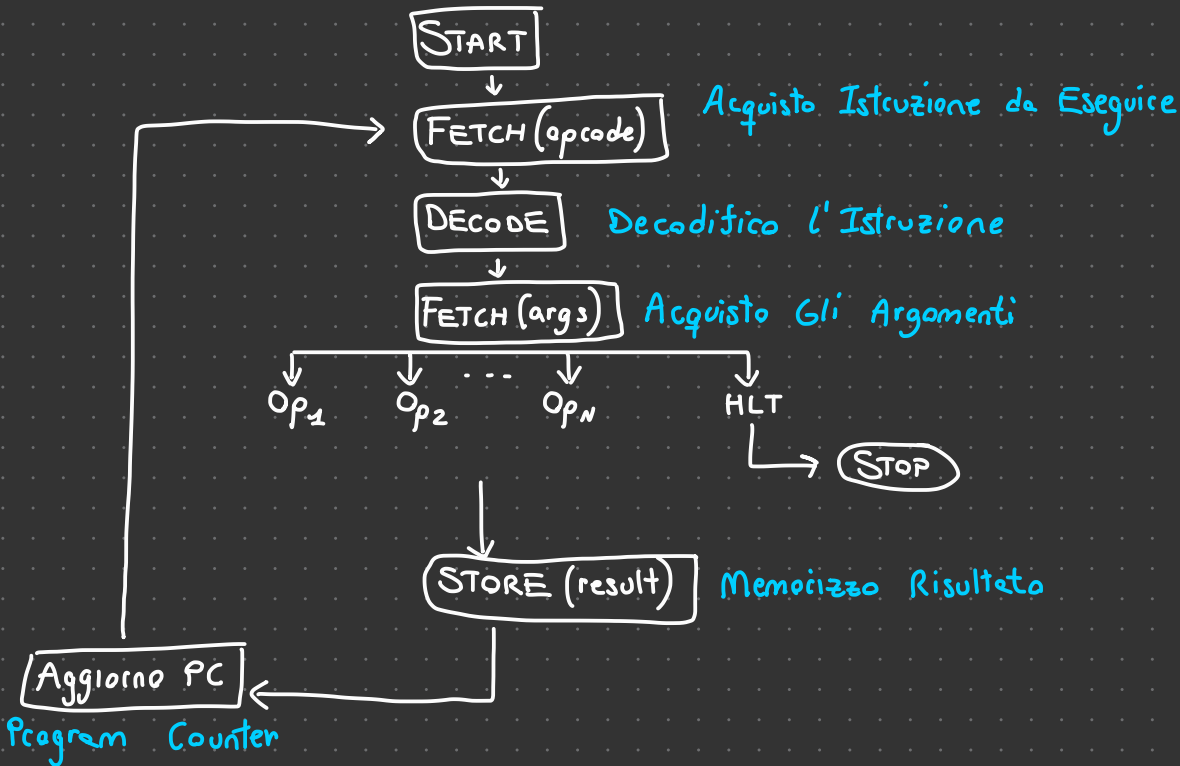
- /// Associa un TIPO All' Espressione
- /// Associa un AMBIENTE STATICO per dichiarazioni
- /// Comando Sia BEN FORMATO

TEORIA:

INTERPRETE per un linguaggio L Scritto in L_0 è un Programma

$$I^{L_0 L} : \text{Prog}^L \times D \rightarrow D \quad \forall \text{input} \in D. I^{L_0 L} (P^L, \text{input}) = \text{output}$$

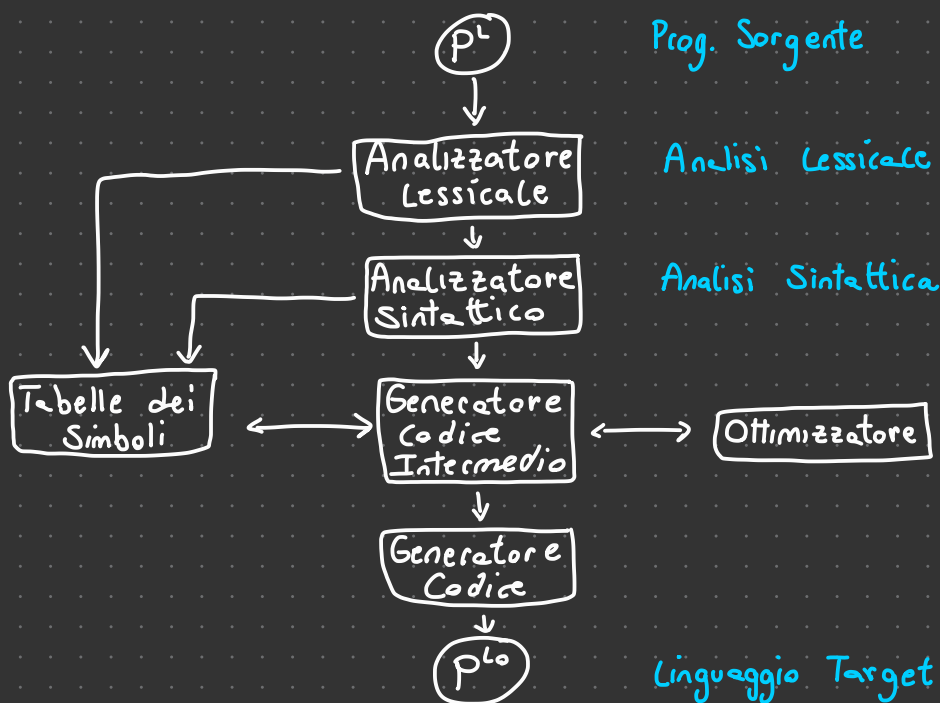
$$\forall P^L \in \text{Prog}^L$$



COMPILATORE da L in L_0 è un Programma "TRADUTTORE" che:
Linguaggio Sorgente $\leftarrow$ $\rightarrow$ Linguaggio Target

$$C^{L_0}: \text{Prog}^L \rightarrow \text{Prog}^{L_0} \text{ t.c.}$$

$$\forall \text{input} \in D \quad \text{Prog}^L(\text{input}) = \text{Prog}^{L_0}(\text{input})$$



CATEGORIE SINTATTICHE

/// **ESPRESSIONI** denotano Valori, Vengono Valutate per Restituire un Valore, Sono Equivalenti se Restituiscono val =

/// **DICHIARAZIONI** denotano Richieste di modifica degli Ambienti, Vengono Elaborate per Ottenere la Trasformazione, Sono Equivalenti Se Producono Stesso Ambiente.

Occorrenze di:

- **DEFINIZIONE** → Si Attribuisce Significato All' Id
- **Uso** → Si Accede al Suo SIGNIFICATO
- **LIBERA** → Non è Nello SCOPE di una definizione

AGGIORNAMENTO AMBIENTE $\Rightarrow \rho[p'] = \rho''$

COMPOSIZIONE:

- SEQUENZIALE $\rightarrow \rho \vdash d_1; d_2 \rightsquigarrow \rho[p_1[p_2]]$
- PRIVATA $\rightarrow \rho \vdash d_1 \text{ IN } d_2 \rightsquigarrow \rho[p_2] \left(\rho_1 \text{ Nascondo all' Esterno} \right)$

/// **COMANDI** denotano Richieste di Modifiche alla MEMORIA, Vengono Eseguiti, Sono **EQUIVALENTI SE PRODUCONO LA STESSA MEMORIA**. (che cattura Trasformazioni Irreversibili)

BINDING:

Se Siamo nel CONTESTO con SCOPING DINAMICO e Viene Passato una fz. Come PARAMETRO con RIFERIMENTI non Locali.

/// **DEEP BINDING** $\Rightarrow$ Ambiente di RIFERIMENTO è Quello della **creazione del legame** Tra Attuale e formale

/// **SHALLOW BINDING** $\Rightarrow$ Ambiente $\swarrow$ Quando Viene Effettuata la **call** è Quello valido

Scoping Statica è SOTTOINTESO come DEEP BINDING

Differenza Tra Shallow e Deep è Come Tra Dinamico e Statico

Solo per i RIFERIMENTI non Locali All' Interno di una Procedura

